



Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

25.08.2026

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Seite 1

Technology
Arts Sciences
TH Köln

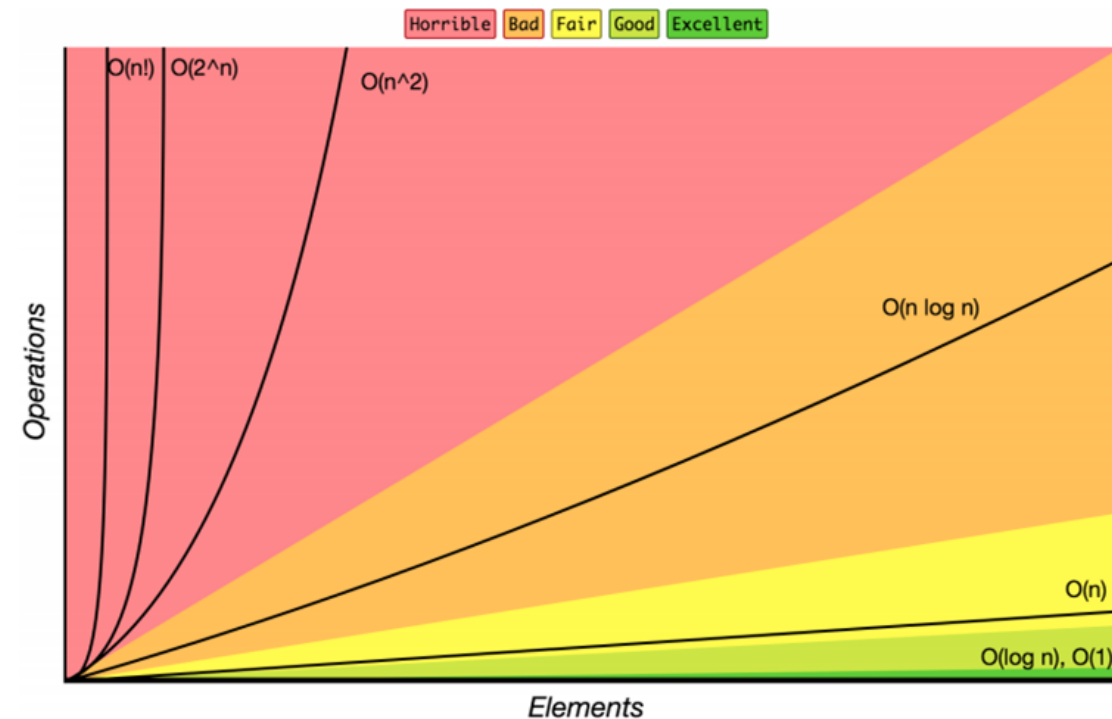
Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
 - Array
 - Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
 - O-Notation
- Laufzeitanalyse von Algorithmen durch Asymptotische Notation
 - O-Notation, Ω -Notation, Θ -Notation
 - Fallanalyse: Worst Case, Best Case Analyse
- Laufzeitanalyse von Rekursiven Algorithmen durch Asymptotische Notation

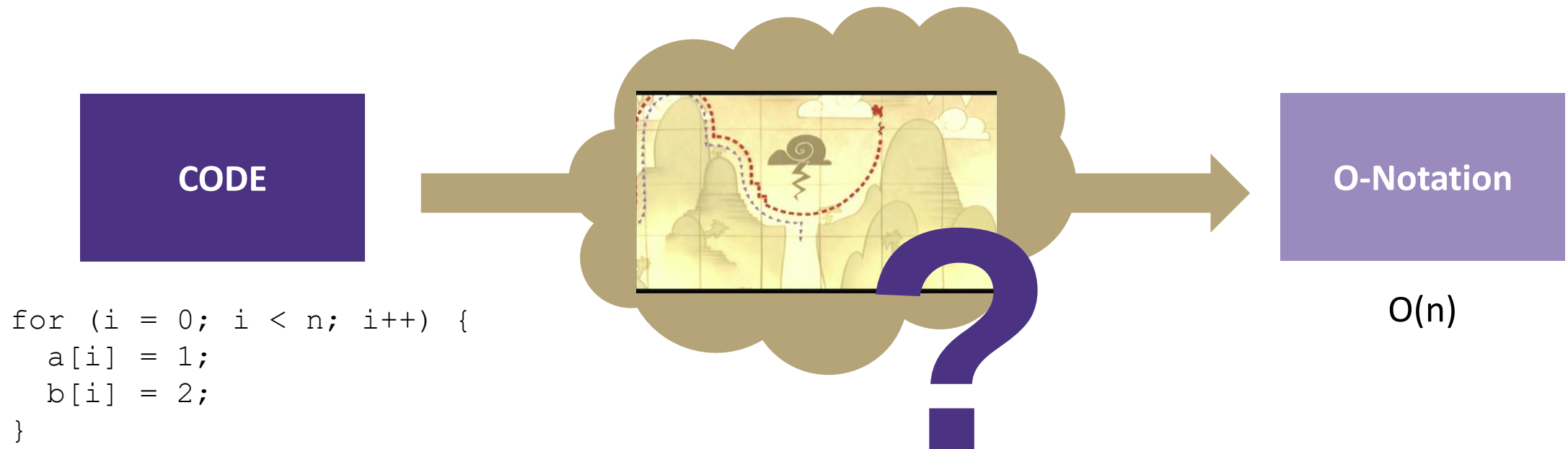
Klassifikation der Wachstumsordnung

Wachstumsordnung: Eine Kategorie der Algorithmeneffizienz, die auf dem Verhältnis des Algorithmus zur Eingabegröße n basiert.

Description	O-Notation	Runtime if you double n	Example Algorithm
constant	$O(1)$	unchanged	Accessing an index of an array
logarithmic	$O(\log_2 n)$	increases slightly	Binary search
linear	$O(n)$	doubles	Looping over an array
log-linear	$O(n \log_2 n)$	slightly more than doubles	Merge sort algorithm
quadratic	$O(n^2)$	quadruples	Nested loops!
...	...	...	...
exponential	$O(2^n)$	multiplies drastically	Fibonacci with recursion



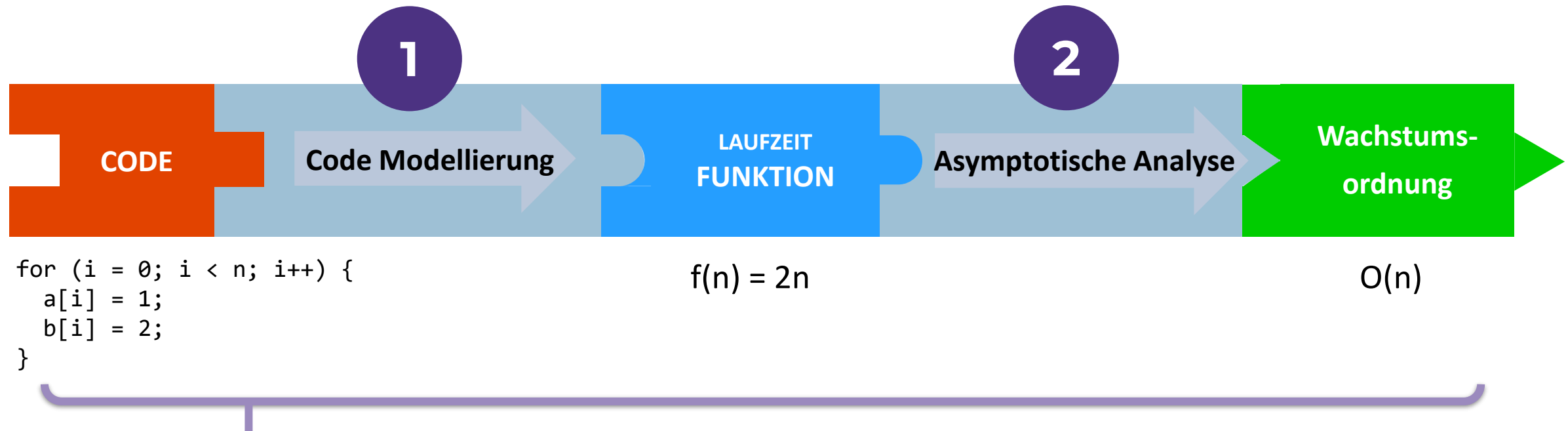
Code in O-Notation umwandeln



Bisher: “ $O(1)$ bedeutet keine Schleifen, $O(n)$ ist eine Schleife, $O(n^2)$ ist verschachtelte Schleife”

- Das ist weiterhin nützlich!
- In diesem Modul gehen wir mehr ins Detail: Damit wir auch komplizierteren Code in die O-Notation umwandeln können

Analyse von Algorithmen

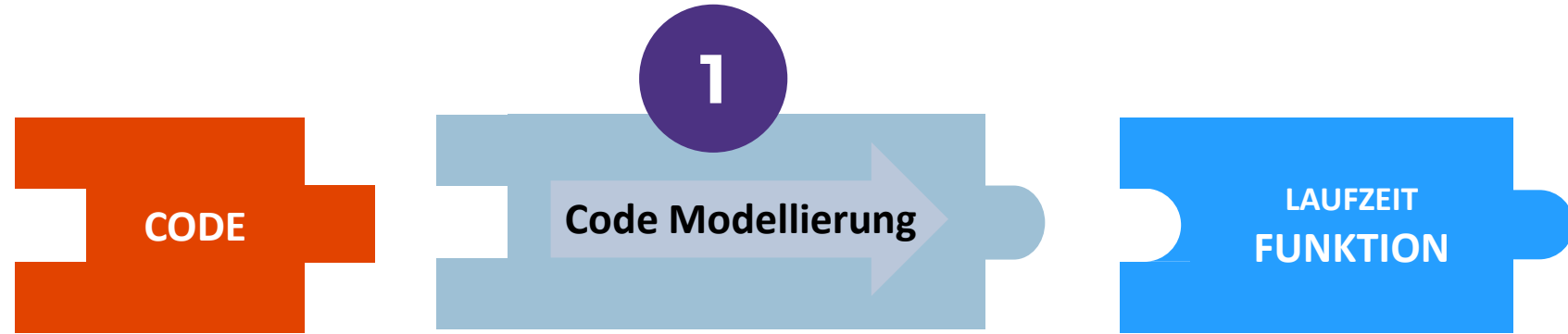


Analyse von Algorithmen: Der Gesamtprozess der Zuordnung von Code zu einer Wachstumsordnung, bestehend aus:

Code Modellierung: Code $\rightarrow$ Laufzeitfunktion, die die Laufzeit des Codes beschreibt

Asymptotische Notation: Funktion $\rightarrow$ Wachstumsordnung, die das asymptotische Verhalten beschreibt

Code Modellierung



Code Modellierung - der Prozess der mathematischen Darstellung, wie viele Operationen ein Stück Code im Verhältnis zur Eingabegröße n ausführt.

- Umwandlung von Code in eine Laufzeitfunktion, die seine Laufzeit darstellt

Code Modellierung

Was zählt wieviel?

Wir kennen nicht die genaue Laufzeit jeder Operation, aber für den Moment wollen wir die vereinfachende Annahme machen, dass alle grundlegenden Operationen die gleiche Zeit benötigen.

Grundlegende Operationen (Laufzeit: 1):

- +, -, /, *, %, ==
- Zuweisung
- Rückgabe
- Zugriff auf Arrays

Funktionsaufrufe

- Gesamtlaufzeit im Funktionskörper
- Zur Erinnerung: `new` ruft eine Funktion auf (Konstruktor)

Bedingte Anweisung (If-Abfrage)

- Bedingung + Zeit für die Verzweigung (if-Block)
- Wir lernen später, wie man Verzweigungen berücksichtigt

Schleifen

- Anzahl der Iterationen * Gesamtlaufzeit in Bedingung und Schleifenkörper

Code Modellierung

Beispiel 1

```
public int method1(int n) {
    int sum = 0;
    int i = 0;
    while (i < n) {
        sum = sum + (i * 3);
        i = i + 1;
    }
    return sum;
}
```

+1

+1

+1

+3

+2

+1

Schleife läuft n-mal

+6

*n

$$f(n) = 6n + 3$$

Code Modellierung

Beispiel 2

```
public int method2(int n) {
    int sum = 0;      +1
    int i = 0;        +1
    while (i < n) {    +1
        int j = 0;    +1
        while (j < n) { +1
            if (j % 2 == 0) { +2
                // do nothing
            }
            sum = sum + (i * 3) + j; +4
            j = j + 1; +2
        }
        i = i + 1; +2
    } return sum; +1
}
```

$$f(n) = (9n+4)n + 3$$

Innere Schleife
läuft n-mal

+9

*n

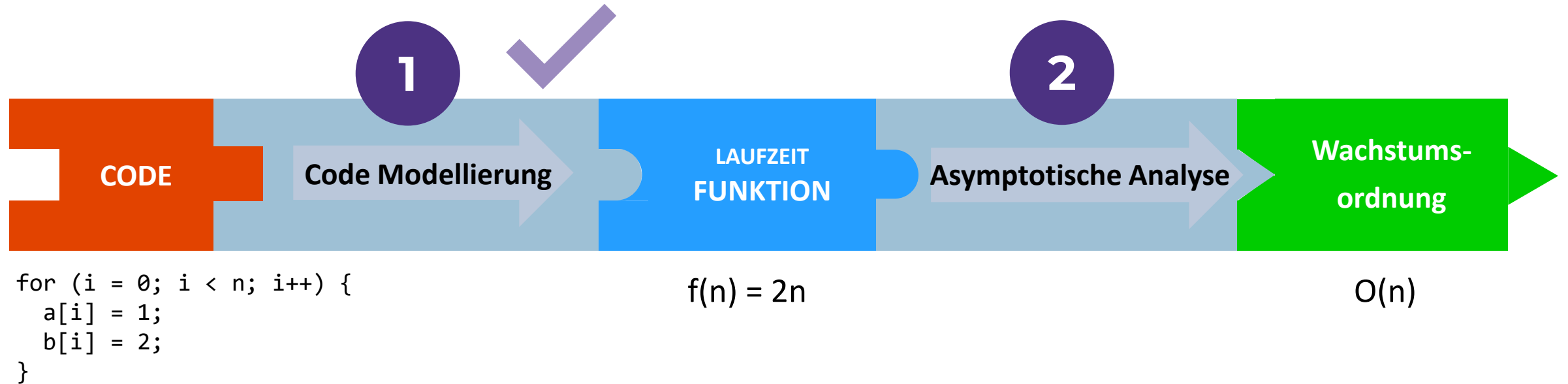
Äußere Schleife
läuft n-mal

9n + 4

*n

Analyse von Algorithmen

Schritt 2



Wir haben gerade ein Stück Code in eine Funktion umgewandelt!

- Wir werden uns später detaillierter mit der Code Modellierung befassen

Jetzt konzentrieren wir uns auf Schritt 2, die asymptotische Analyse

Finden der O-Notation



Wie kommen wir von der Laufzeitfunktion $f(n)$ auf die O-Notation?

1. Finde den dominierenden Term und lösche alle anderen.
 - Der dominierende Term ist der größte Term, wenn n größer wird.
Meistens die höchste Potenz von n .
2. Lösche alle konstanten Faktoren.

$$f(n) = (9n+4)n + 3$$

$$= 9n^2 + 4n + 3$$

$$\approx 9n^2$$

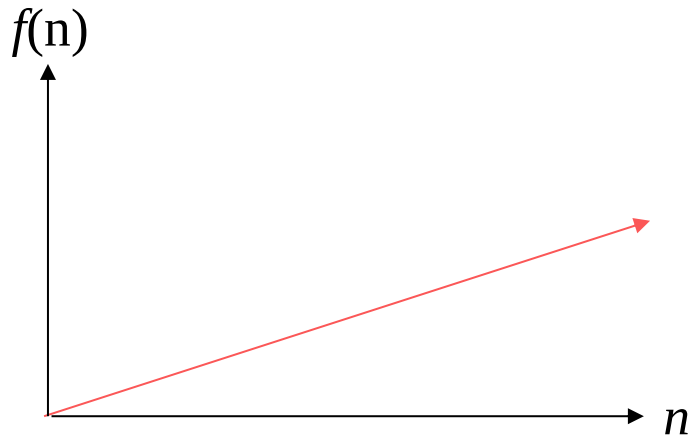
$$\approx n^2$$

$f(n)$ ist in $O(n^2)$

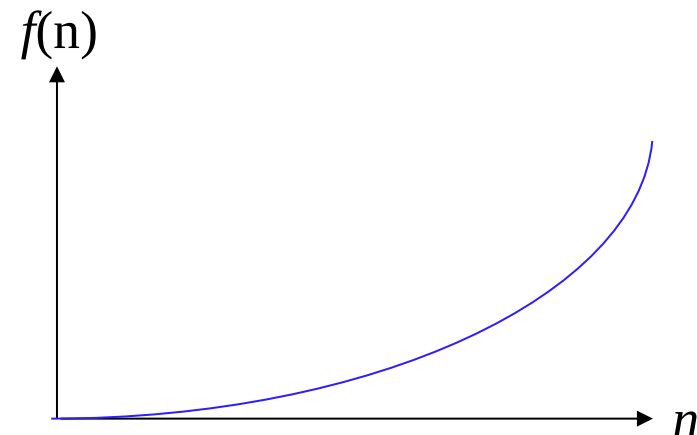
Was ist ein „dominierender Term“?

Dominierende Terme haben den größten Einfluss auf das Verhalten von $f(n)$, wenn n gegen Unendlich geht, da sie dann die größten sind und die kleineren Terme „dominieren“

$$\underline{f(n) = n + 10}$$



$$\underline{f(n) = n^2 + n + 10}$$



Was ist ein „dominierender Term“?

Welcher ist der dominierende Term?

- | | |
|---------------------------------|-----------|
| 1. $n^2 + n$ | n^2 |
| 2. $n + 1000$ | n |
| 3. $n^{100} + n^{50} + n^2 + 5$ | n^{100} |
| 4. $n^2 + 2^n$ | 2^n |
| 5. $3^n + 4^n$ | 4^n |

Hinweis: Fragen Sie sich: „Welcher Term wird am größten sein, wenn n immer größer wird?“

Finden der O-Notation

Können wir wirklich die ganzen Terme wegwerfen?

Asymptotische Analyse: Analyse des Verhaltens einer Funktion bei Annäherung ihrer Eingabe an die Unendlichkeit

- Wir interessieren uns nur dafür, was passiert, wenn n sich der Unendlichkeit nähert
- Bei kleinen Eingaben spielt Laufzeit keine Rolle: Jeder Code ist „schnell genug“.
- Da wir es mit unendlich zu tun haben, tragen Konstanten und Terme niedrigerer Ordnung nicht wesentlich zum Endergebnis bei. Der Term höchster Ordnung ist das, was das Wachstum antreibt!

Unsere Ziele waren:

1.

Einfach

Winzige Unterschiede in der Umsetzung sind uns egal, wir wollen das „Big Picture“

2.

Ausschlaggebend

Erstellen eines klaren Vergleichs, der zeigt, welcher Code „länger“ dauert

Wachstumsordnungen

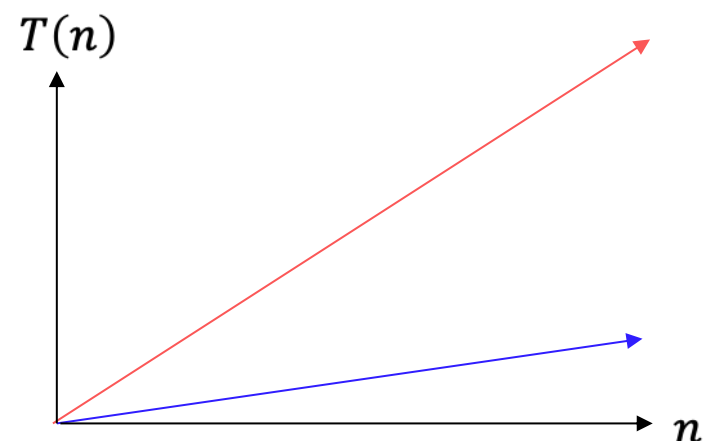
Stellen Sie sich vor, Sie haben drei mögliche Algorithmen zur Auswahl.

Jeder wurde bereits auf seine Laufzeitfunktion reduziert

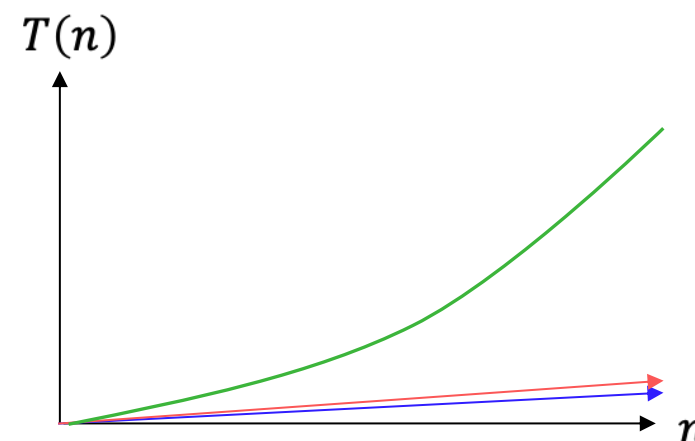
$$\underline{f(n) = n}$$

$$\underline{g(n) = 4n}$$

$$\underline{h(n) = n^2}$$



Die Wachstumsrate für $f(n)$ und $g(n)$ sieht für kleine Eingangsgrößen sehr unterschiedlich aus



...aber da beide linear sind, sehen sie bei großen Eingangsgrößen ähnlich aus. während $h(n)$ eine deutlich andere Wachstumsrate aufweist

Definition: O-Notation

Bestimmung der O-Notation erfolgt durch

- Folgen der Handlungsanweisungen (Terme niedriger Ordnung löschen, konstante Faktoren löschen)
- Anwenden der Definition der O-Notation

O-Notation

$f(n) \in O(g(n))$ falls positive Konstanten

c und n_0 existieren, sodass

$$0 \leq f(n) \leq c \cdot g(n) \text{ für alle } n \geq n_0$$

Bedeutung der O-Notation:

- Die Laufzeit, beschrieben durch die Laufzeitfunktion $f(n)$, beträgt für n gegen unendlich maximal $c \cdot g(n)$.
- $c \cdot g(n)$ ist eine obere Schranke von $f(n)$

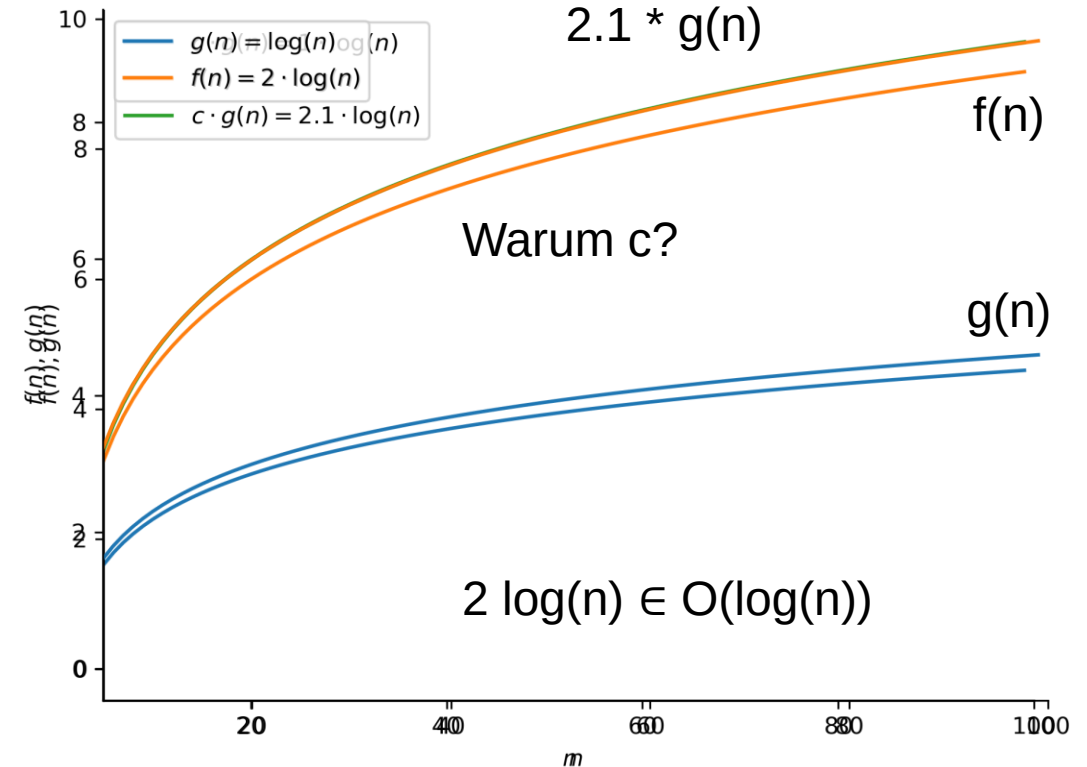
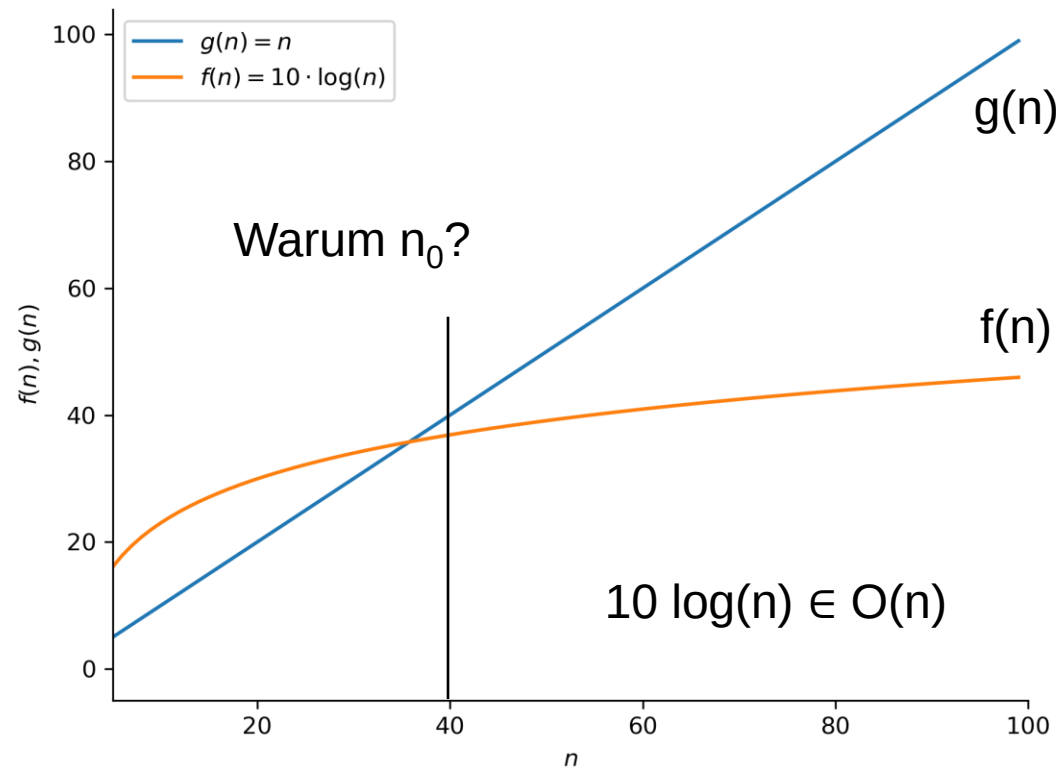
Laufzeitfunktion: $f(n) = 9n^2 + 4n + 3$

Wachstumsordnung: $g(n) = n^2$

Definition: O-Notation

O-Notation

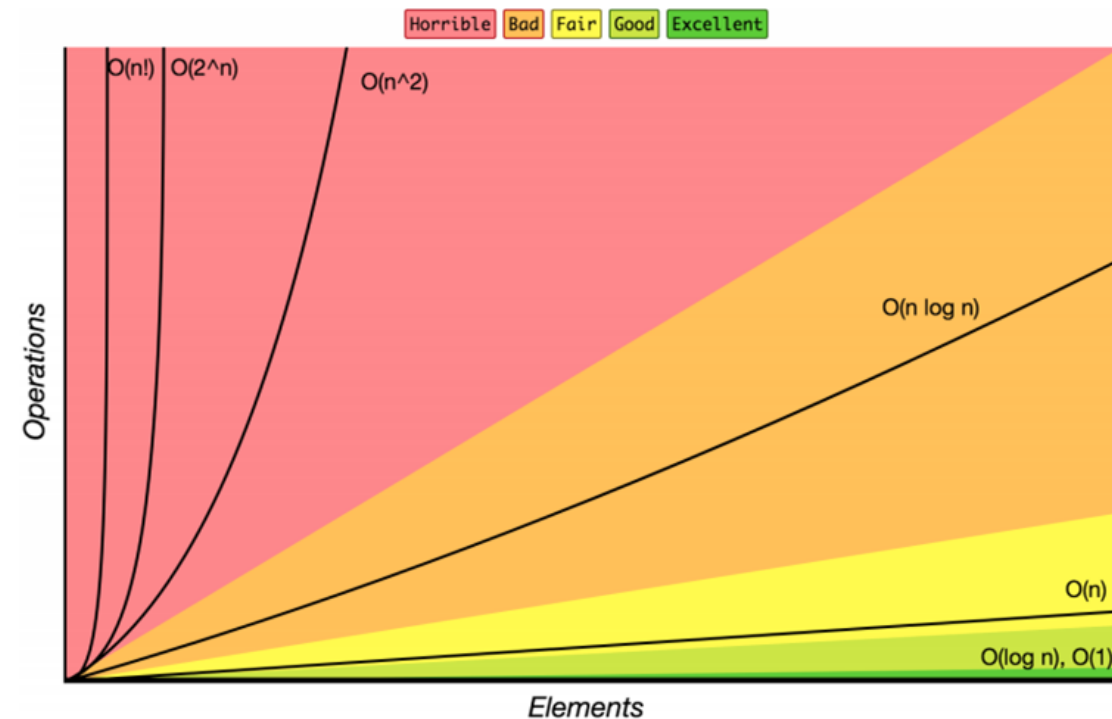
$f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$



Klassifikation der Wachstumsordnung

Wachstumsordnung: Eine Kategorie der Algorithmeneffizienz, die auf dem Verhältnis des Algorithmus zur Eingabegröße n basiert.

Description	O-Notation	Runtime if you double n	Example Algorithm
constant	$O(1)$	unchanged	Accessing an index of an array
logarithmic	$O(\log_2 n)$	increases slightly	Binary search
linear	$O(n)$	doubles	Looping over an array
log-linear	$O(n \log_2 n)$	slightly more than doubles	Merge sort algorithm
quadratic	$O(n^2)$	quadruples	Nested loops!
...	...	...	...
exponential	$O(2^n)$	multiplies drastically	Fibonacci with recursion



Fragen?

- Code Modellierung
- O-Notation

O-Notation anwenden

Beispiel 1

Zeigen Sie, dass $f(n) \in O(n)$ für $f(n) = 10n + 15$

Definition Term für Term anwenden

$$0 \leq 10n \leq c \cdot n \text{ für } c = 10, n \geq 0$$

$$0 \leq 15 \leq c \cdot n \text{ für } c = 15, n \geq 1$$

Alle Aussagen addieren und maximales n_0 wählen

$$0 \leq 10n + 15 \leq 10n + 15n = 25n \text{ für } n \geq 1$$

Wähle Werte für c und n_0 und zeige, dass diese der Definition genügen

Für $c = 25$ und $n_0 = 1$ gilt also:

$$0 \leq 10n + 15 \leq 25n \text{ für alle } n \geq 1$$

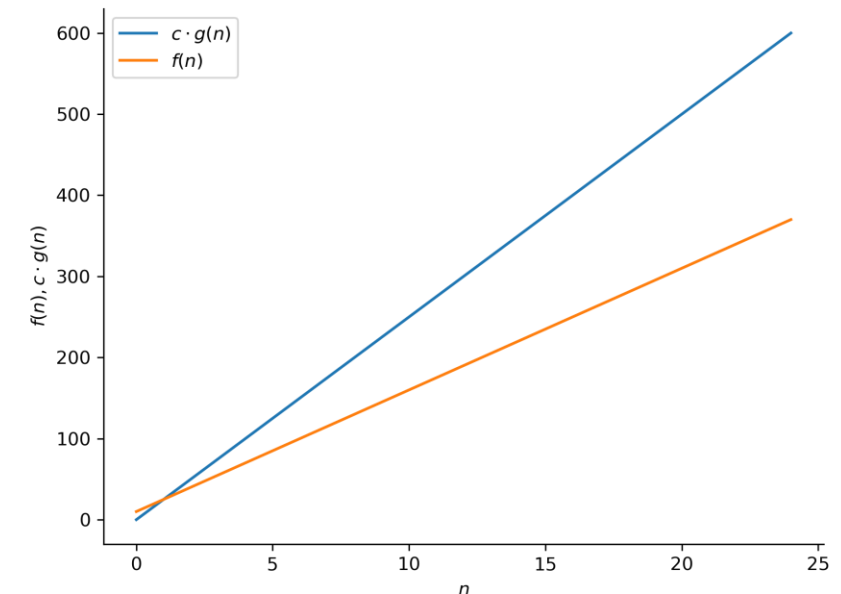
Da es ein c und n_0 gibt, gilt $f(n) \in O(n)$

O-Notation

$f(n) \in O(g(n))$ falls positive Konstanten

c und n_0 existieren, sodass

$$0 \leq f(n) \leq c \cdot g(n) \text{ für alle } n \geq n_0$$



Die O-Notation ist nur eine obere Schranke, nicht immer eine exakte Schranke

Es gibt viele obere Schranken für $f(n)$.

Sie könnten prinzipiell fast immer sagen, dass die Laufzeit in $O(n!)$ sei (faktorielle Laufzeit).

Die informativste obere Schranke ist die kleinste obere Schranke im Rahmen der Definition der O-Notation.

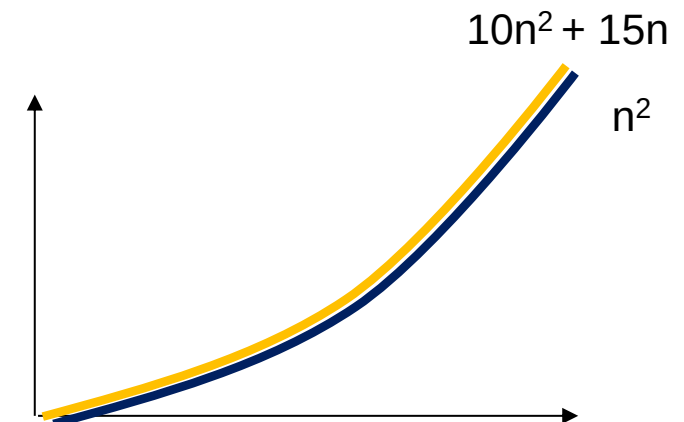
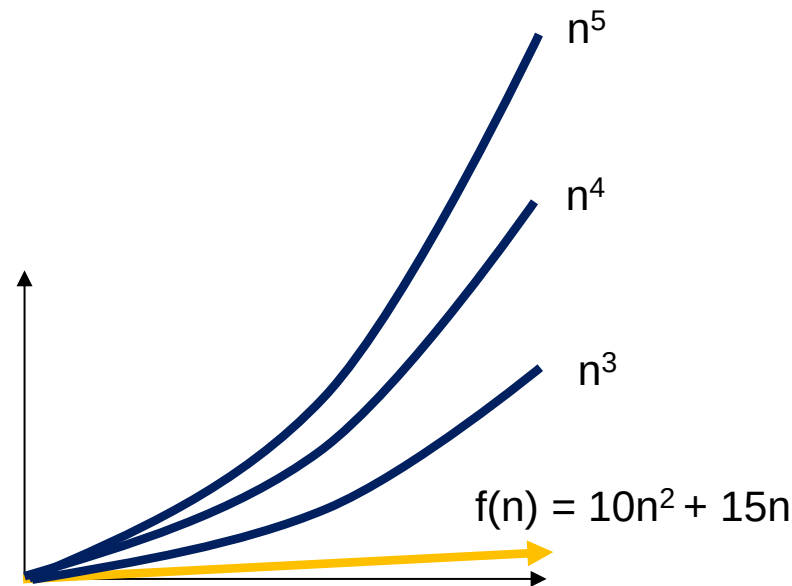
Nach dieser ist immer gesucht, hier $O(n^2)$.

$$10n^2 + 15n \in O(n^3)$$

$$10n^2 + 15n \in O(n^4)$$

$$10n^2 + 15n \in O(n^5)$$

... und so weiter ...



Fragen?

- Definition der O-Notation anwenden

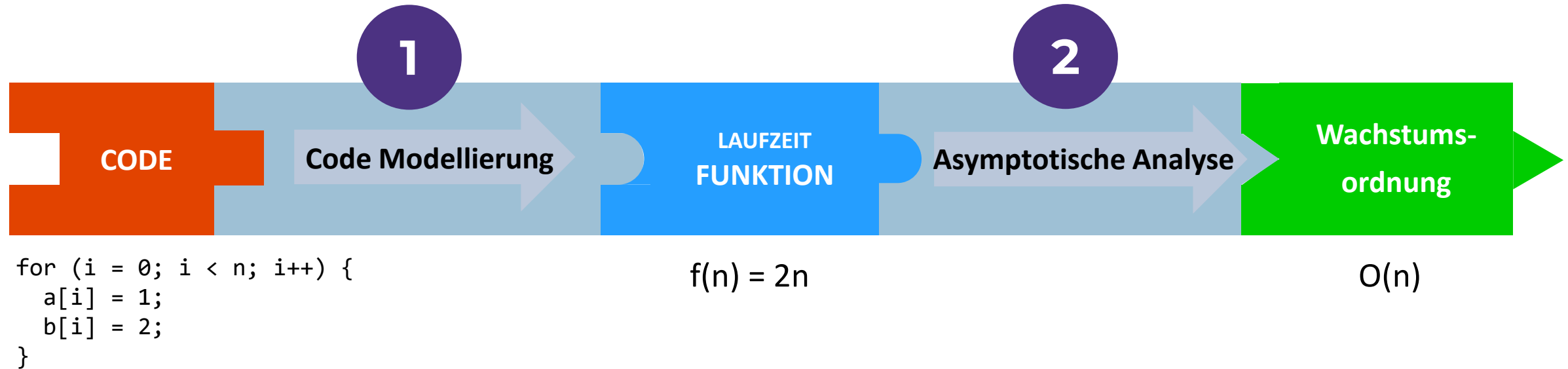
Was Sie sich merken sollten:

- Bestimmung der O-Notation für eine Laufzeitfunktion $f(n)$ in zwei Schritten:
 - (1) Finde den dominierenden Term und lösche alle anderen.
 - (2) Lösche alle konstanten Faktoren.
- Die O-Notation gibt nur eine obere Schranke für die asymptotische Laufzeit an

Übungsblatt 2: Übung 1 und 2

Analyse von Algorithmen

Schritt 2 – Asymptotische Analyse für komplexeren Code

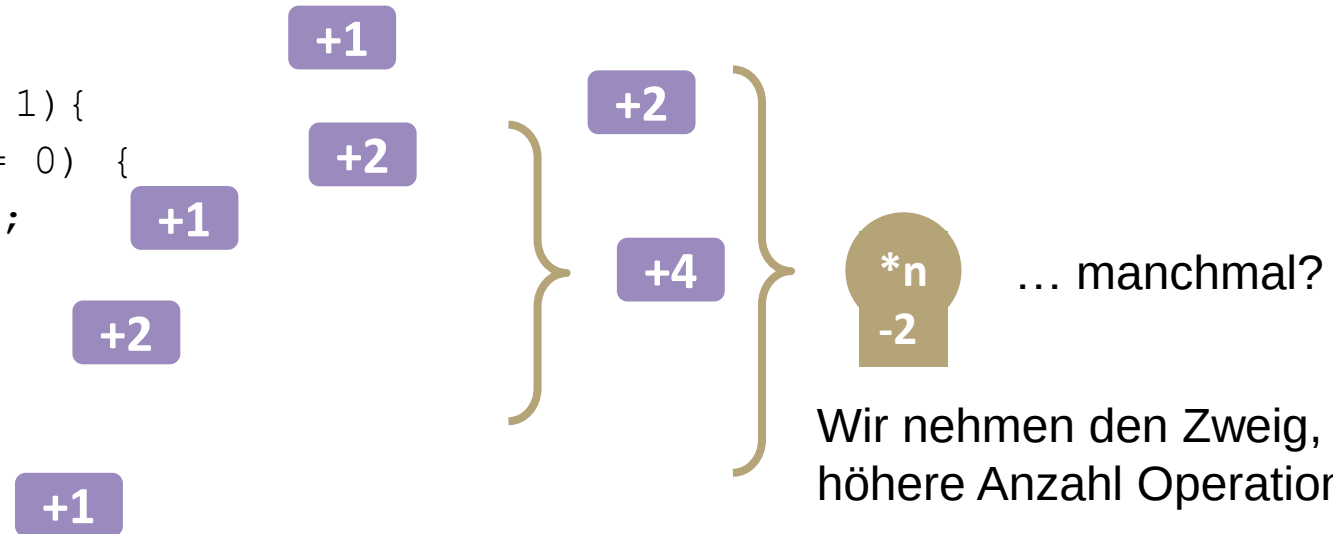


Asymptotische Analyse für komplexere Funktionen

Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2023

Finden Sie die Laufzeitfunktion $f(n)$ dieses Codes bei Eingabe n . Wie lautet die O-Notation?

```
boolean isPrime(int n){
    int curDiv = 2;
    while(curDiv <= n - 1){
        if(n % curDiv == 0) {
            return false;
        } else {
            curDiv++;
        }
    }
    return true;
}
```



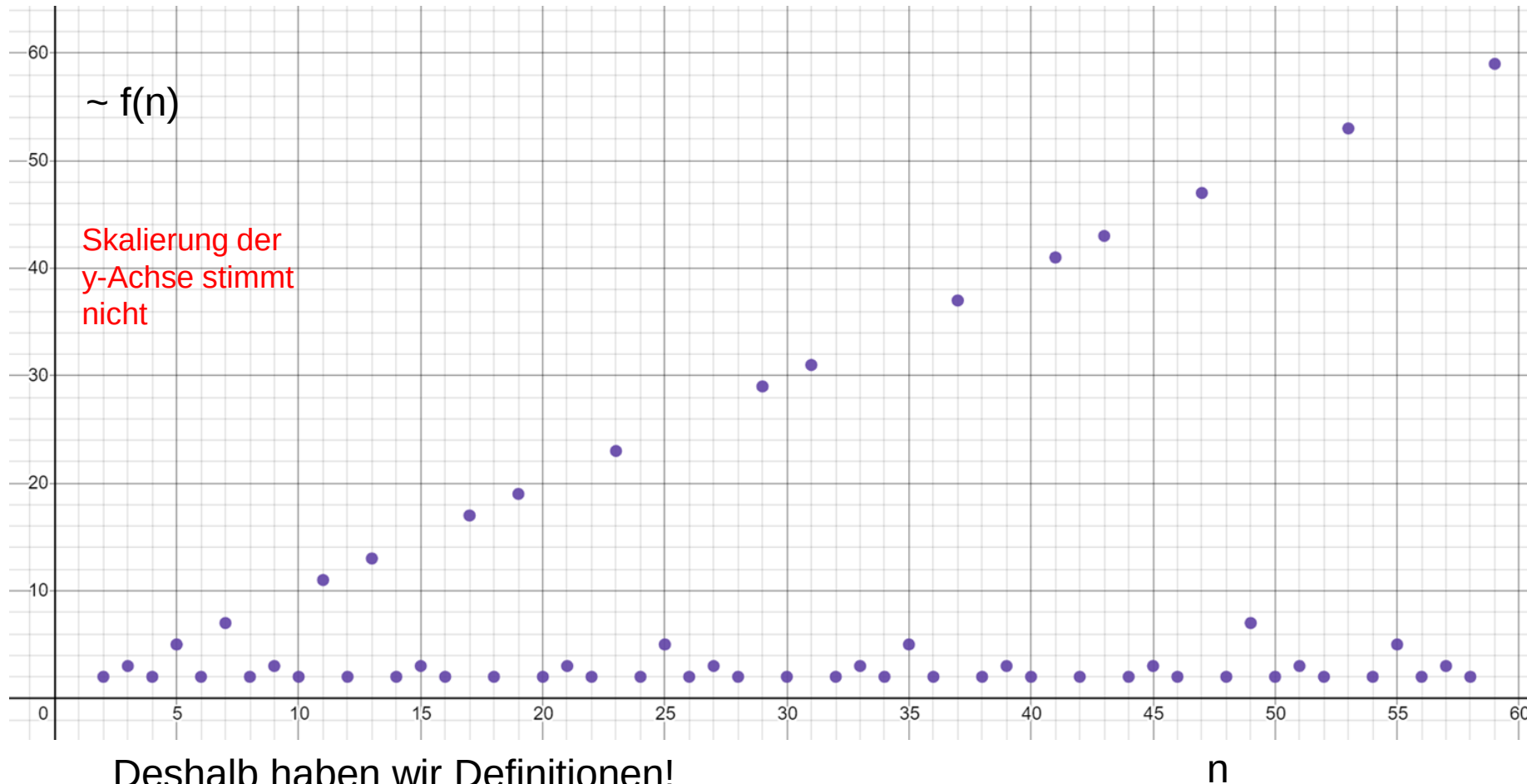
Wir nehmen den Zweig, der die höhere Anzahl Operationen hat

$$f(n) = 6(n-2)+2$$

Suche nach dem kleinsten Teiler von n

Asymptotische Analyse für komplexere Funktionen

Laufzeit von isPrime()

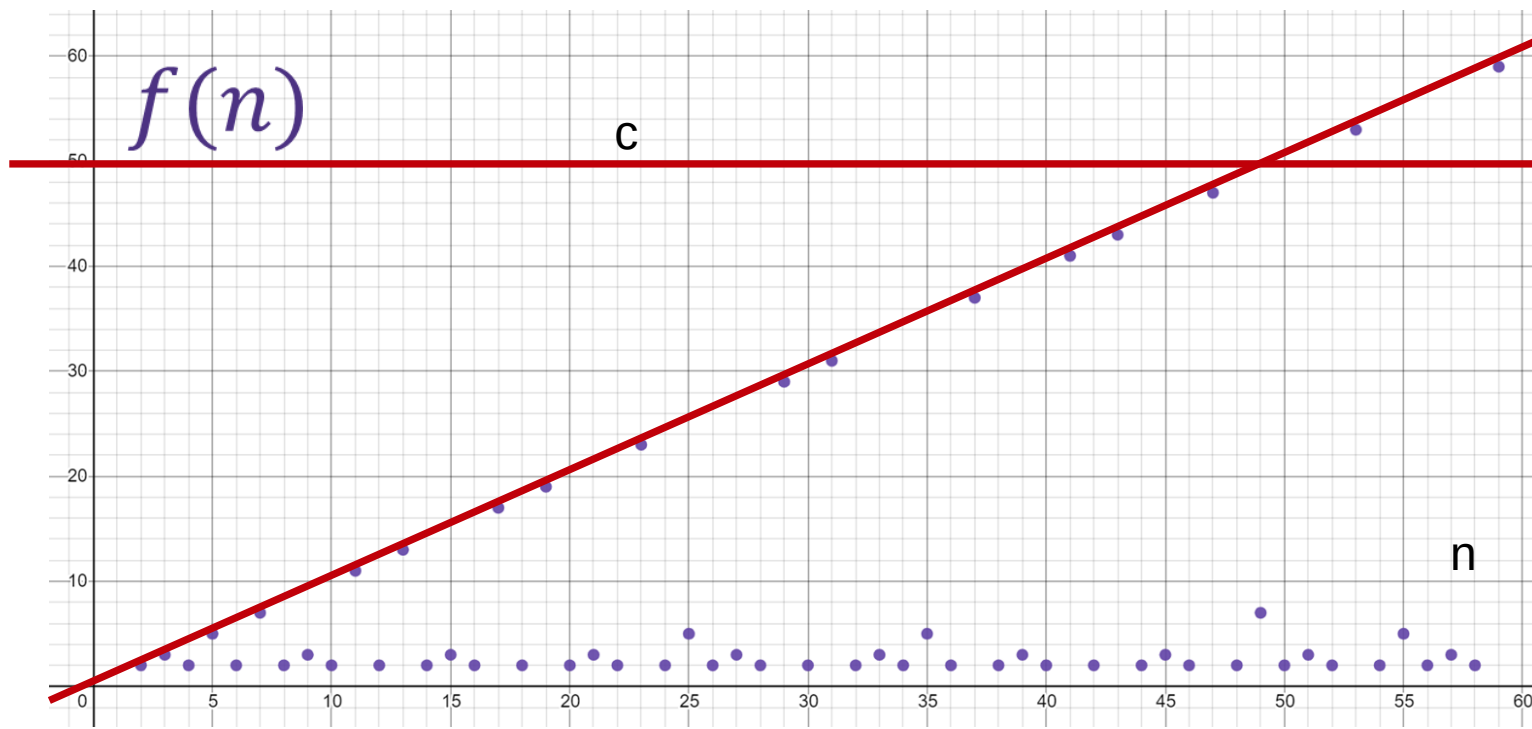


Ist die Laufzeit in $O(1)$ oder in $O(n)$?

Mehr als die Hälfte der Fälle benötigen wir unter 10 Operationen. Ist es $O(1)$?

Allerdings, für manche n benötigt der Code $f(n)$ Operationen. Also $O(n)$?

Deshalb haben wir Definitionen!



O-Notation

$f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$

Ist die Laufzeit in $O(n)$? Finden Sie positive Konstanten c und n_0 ?

Bspw. für $c = 7$ und $n_0 = 2$ gilt:

$$0 \leq f(n) \leq 7 \cdot n \text{ für alle } n \geq 2$$

$$f(n) \in O(n) \text{ und } f(n) \notin O(1)$$

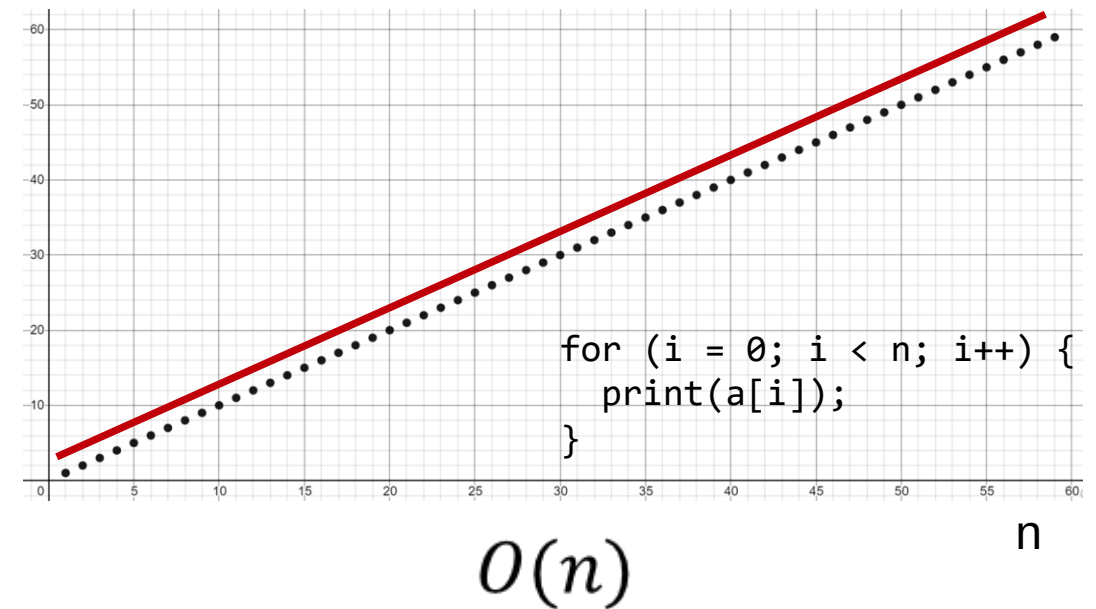
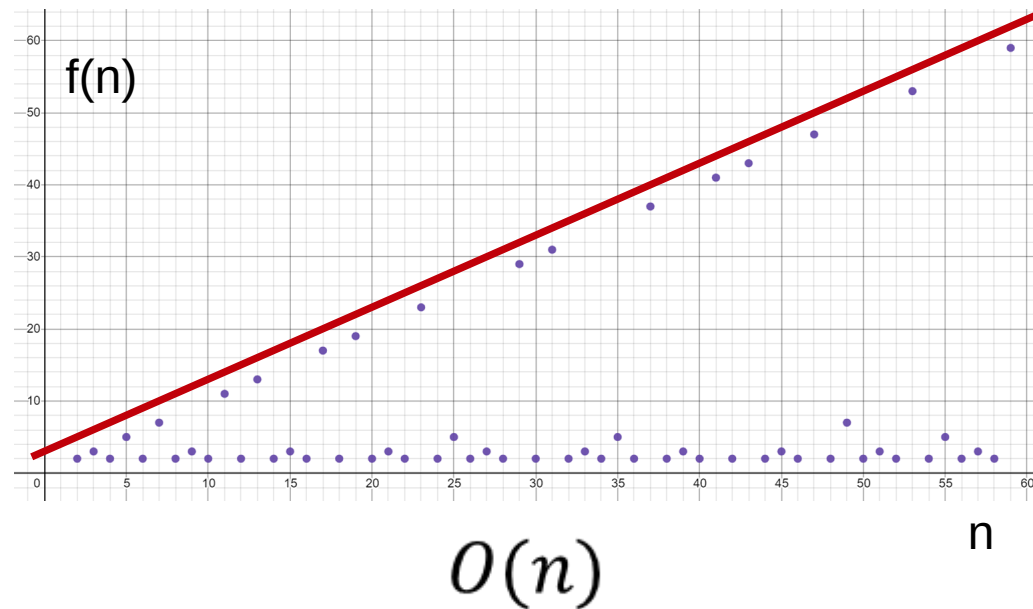
Ist die Laufzeit in $O(1)$? Finden Sie positive Konstanten c und n_0 ?

Nein! Für ein beliebiges c , gibt es eine Primzahl k größer als c .

Und $f(k) = 6 \cdot k + 2 > c \cdot 1$, deshalb hält die Definition nicht.

Die O-Notation ist nicht alles

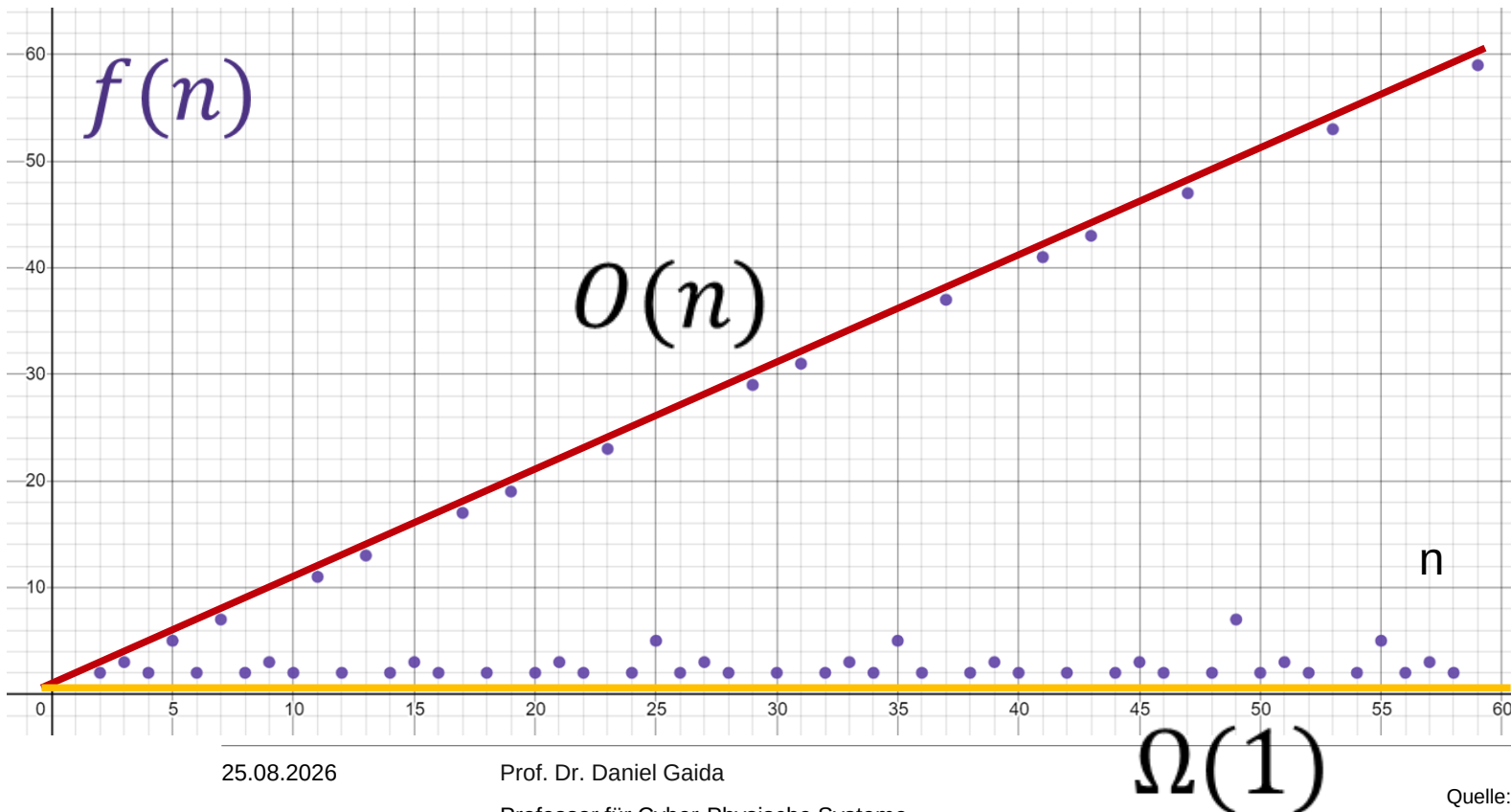
Code zum Auffinden von Primzahlen ist $O(n)$. Gilt z.B. auch für das Ausgeben aller Elemente einer Liste.



Für manche n 's ist die Laufzeit der linken Funktion deutlich kleiner als der rechte Code. Mit der O-Notation alleine kann dieser Umstand nicht beschrieben werden. Können wir darauf hinweisen, dass der rechte Code immer MINDESTENS n Operationen benötigt?

Ω-Notation

Visuell: Was ist die untere Schranke der Laufzeit dieser Funktion? Wodurch wird sie nach unten begrenzt?



Ω-Notation

$f(n) \in \Omega(g(n))$ falls positive Konstanten c und n_0 existieren, sodass $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$

- Die formale Definition von Ω ist die umgekehrte Version von O .
- Wenn wir sagen, $f(n) \in O(g(n))$, sagen wir damit, dass $f(n)$ **höchstens** so schnell wächst wie $g(n)$.
- Aber mit Aussagen wie $f(n) \in \Omega(g(n))$ sagen wir, dass $f(n)$ **mindestens** so schnell wächst wie $g(n)$.

Asymptotische Analyse für komplexere Funktionen

Finden Sie die Laufzeitfunktion $f(n)$ dieses Codes bei Eingabe n . Wie lautet die O-Notation?

```
boolean isPrime(int n){
    int curDiv = 2;
    while(curDiv <= n - 1){
        if(n % curDiv == 0) {
            return false;
        } else {
            curDiv++;
        }
    }
    return true;
}
```

+1

+2

+1

+2

$f(n) \in O(n)$

Wenn n eine Primzahl ist:

$$f(n) = 6(n-2)+2$$

Wenn n keine Primzahl ist,
bspw. n gerade:

$$f(n) = 4$$

Für gewisse n wird die Laufzeitfunktion $f(n)$ also nach unten durch eine konstante Funktion begrenzt, also $f(n) \in \Omega(1)$.

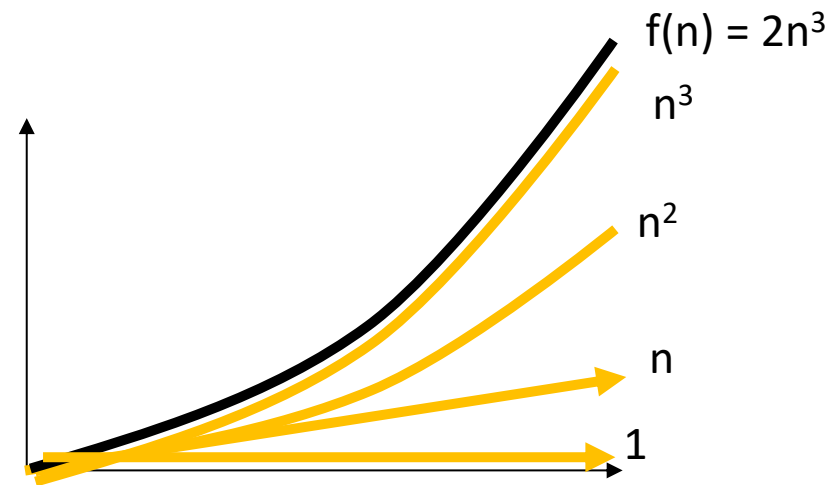
Ω -Notation

$$2n^3 \in \Omega(1)$$

$$2n^3 \in \Omega(n)$$

$$2n^3 \in \Omega(n^2)$$

$$2n^3 \in \Omega(n^3)$$



Die Laufzeitfunktion $f(n) = 2n^3$ wird von allen oben genannten Wachstumsordnungen (1 , n , n^2 , n^3) nach unten begrenzt.

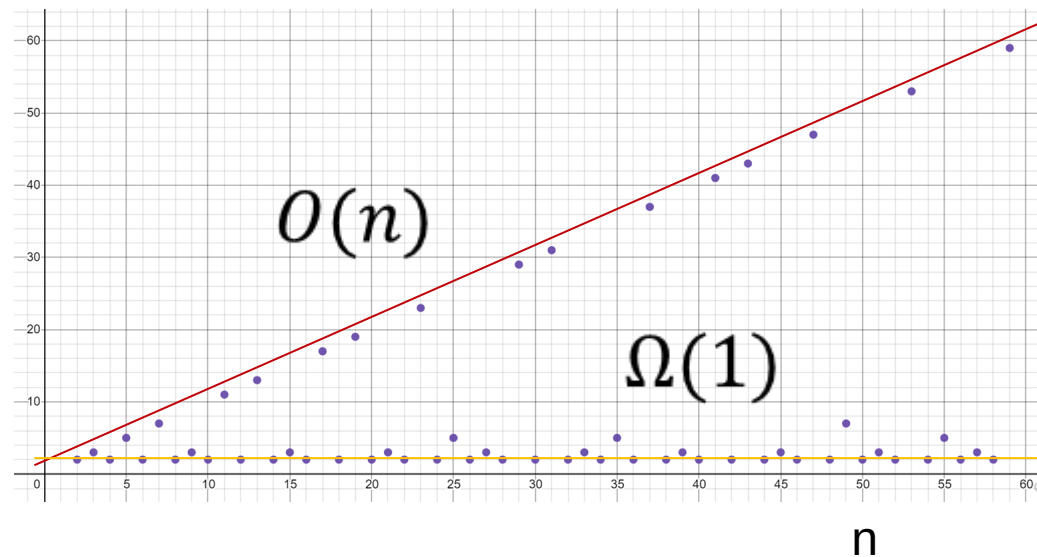
Für dieses $f(n)$ ist $\Omega(n^3)$ wiederum die informativste untere Schranke, da sie die größte untere Schranke ist.

O-Notation und Ω -Notation

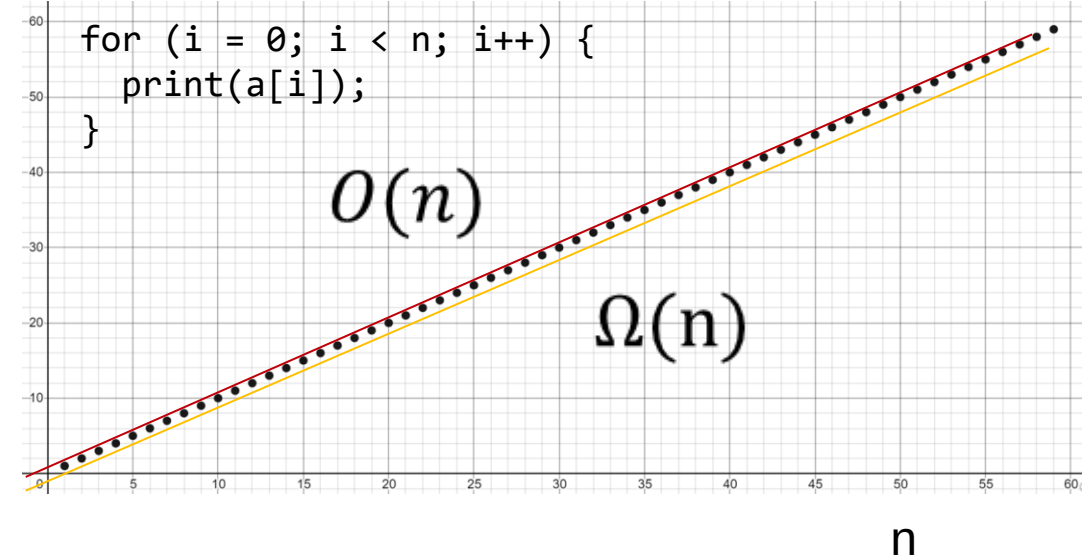
Die O-Notation dieses rechten Graphen ist $O(n)$ und seine Ω -Notation ist $\Omega(n)$.

So sehen die meisten Funktionen aus, mit denen wir uns befassen werden, aber es gibt auch Code, der Laufzeitfunktionen wie die linke erzeugen würde.

Laufzeit der isPrime() Funktion



$f(n) = n$



O-, Ω - und Θ -Notation

Θ -Notation bedeutet:

- Code braucht „genau“*
so lange zur Ausführung
- *Abgesehen von konstanten
Faktoren und Termen niedrigerer Ordnung

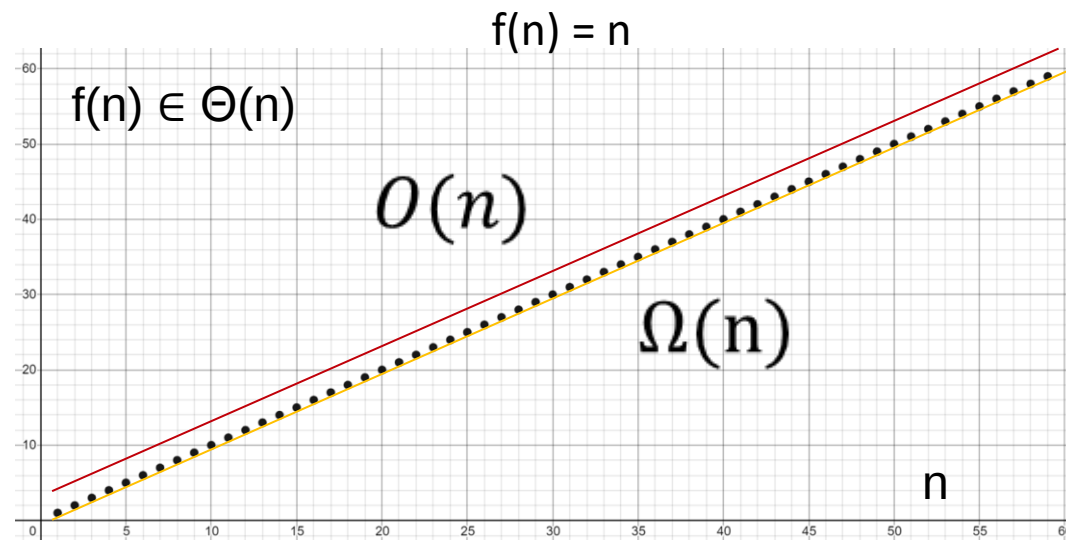
Θ -Notation

$f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.

$f(n) \in \Theta(g(n))$ falls positive Konstanten

c_1, c_2 und n_0 existieren, sodass

$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ für alle $n \geq n_0$



Um die Θ -Notation definieren zu können, müssen sich die O -Notation und die Ω -Notation im Graphen berühren (d. h. sie gehören derselben Wachstumsordnung an)

O-, Ω- und Θ-Notation

O-Notation ist eine obere Schranke

- Code braucht höchstens so lange für die Ausführung

Ω-Notation ist eine untere Schranke

- Code braucht mindestens so lange für die Ausführung

Θ-Notation bedeutet:

- Code braucht „genau“* so lange für die Ausführung
- *Abgesehen von konstanten Faktoren und Termen niedriger Ordnung

O-Notation

$f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$

Ω-Notation

$f(n) \in \Omega(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$

Θ-Notation

$f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.
 $f(n) \in \Theta(g(n))$ falls positive Konstanten c_1, c_2 und n_0 existieren, sodass
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ für alle $n \geq n_0$

O-, Ω- und Θ-Notation

Beispiel: Laufzeitfunktion $f(n) = 4n^2$

$$4n^2 \in \Omega(1)$$

wahr

$$4n^2 \in \Omega(n)$$

wahr

$$4n^2 \in \Omega(n^2)$$

wahr

$$4n^2 \in \Omega(n^3)$$

falsch

$$4n^2 \in \Omega(n^4)$$

falsch

$$4n^2 \in O(1)$$

falsch

$$4n^2 \in O(n)$$

falsch

$$4n^2 \in O(n^2)$$

wahr

$$4n^2 \in O(n^3)$$

wahr

$$4n^2 \in O(n^4)$$

wahr

$$4n^2 \in \Theta(n^2)$$

O-Notation

$f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$

Ω-Notation

$f(n) \in \Omega(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$

Θ-Notation

$f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.
 $f(n) \in \Theta(g(n))$ falls positive Konstanten c_1, c_2 und n_0 existieren, sodass
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ für alle $n \geq n_0$

O-, Ω - und Θ -Notation

Übung

Welche der folgenden Funktionen ist in $O(n^2)$? $\Omega(n^2)$? $\Theta(n^2)$?

a. $f(n) = 42$

$f(n) \in O(n^2)$

b. $f(n) = 5n + 100$

$f(n) \in O(n^2)$

c. $f(n) = n \log_2(3n)$

$f(n) \in O(n^2)$

d. $f(n) = 4n^2 - 2n + 10$

$f(n) \in O(n^2)$ $f(n) \in \Omega(n^2)$ $f(n) \in \Theta(n^2)$

e. $f(n) = 2^n$

$f(n) \in \Omega(n^2)$

O-Notation

$f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$

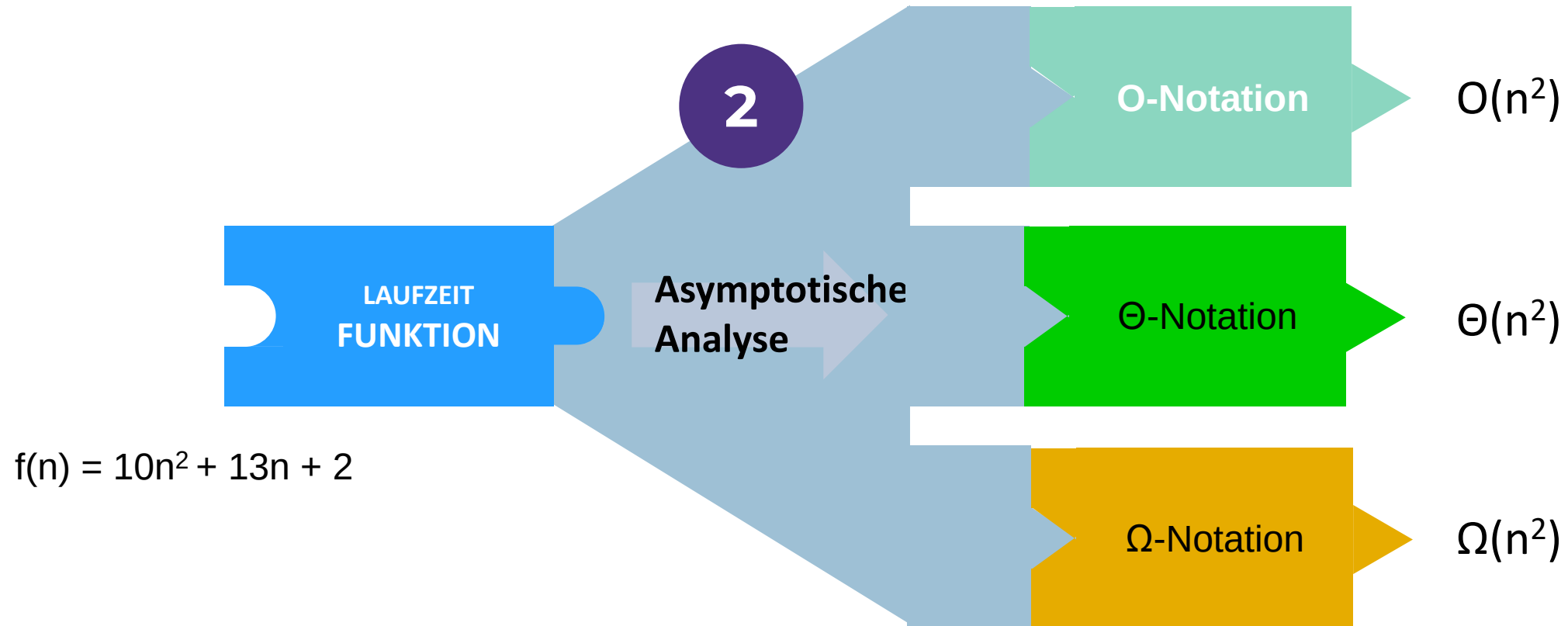
Ω -Notation

$f(n) \in \Omega(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$

Θ -Notation

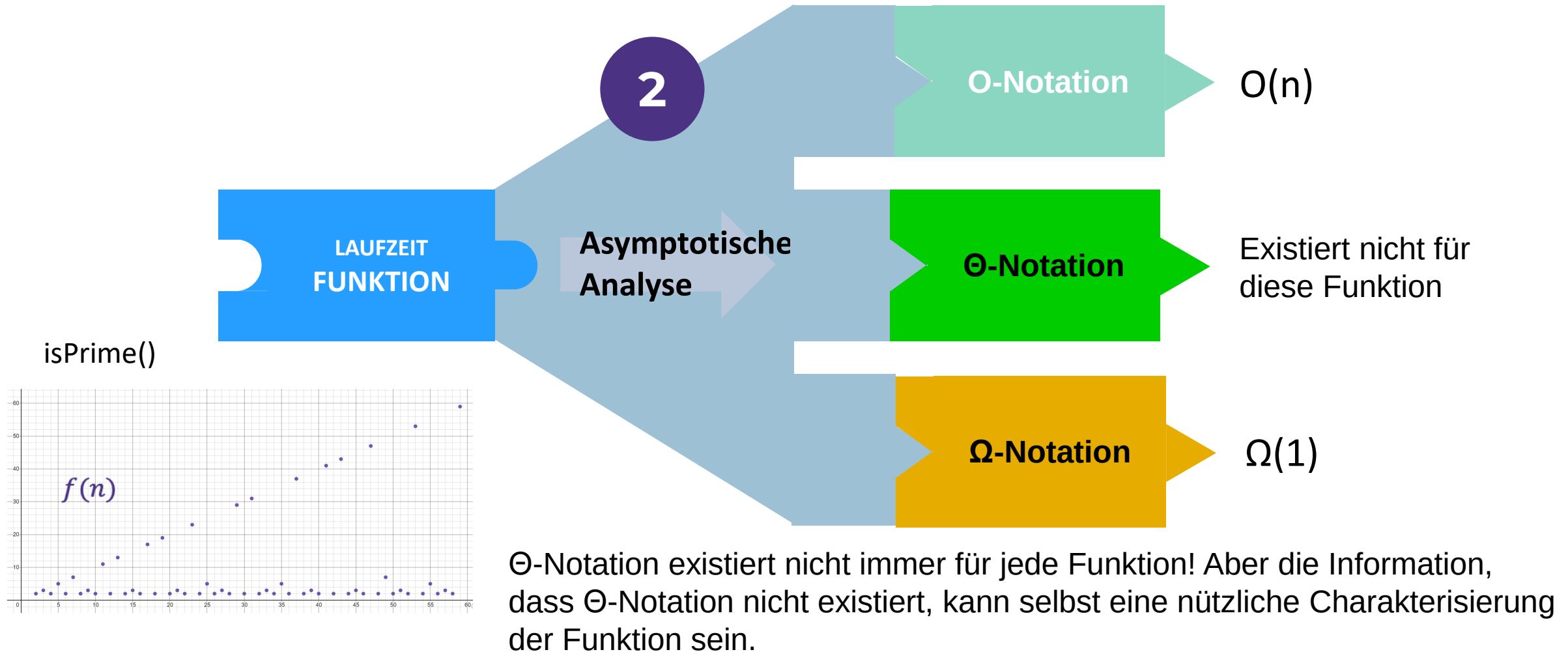
$f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.
 $f(n) \in \Theta(g(n))$ falls positive Konstanten c_1, c_2 und n_0 existieren, sodass
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ für alle $n \geq n_0$

Unser verbessertes Werkzeug: Asymptotische Analyse



Wir haben unser Werkzeug für die Asymptotische Analyse verbessert, um mehr nützliche Informationen zu vermitteln! Mit 3 verschiedenen Arten von Grenzen können wir die Laufzeitfunktion immer noch in einfachen Termen charakterisieren, aber sie gründlicher beschreiben als nur mit der O-Notation.

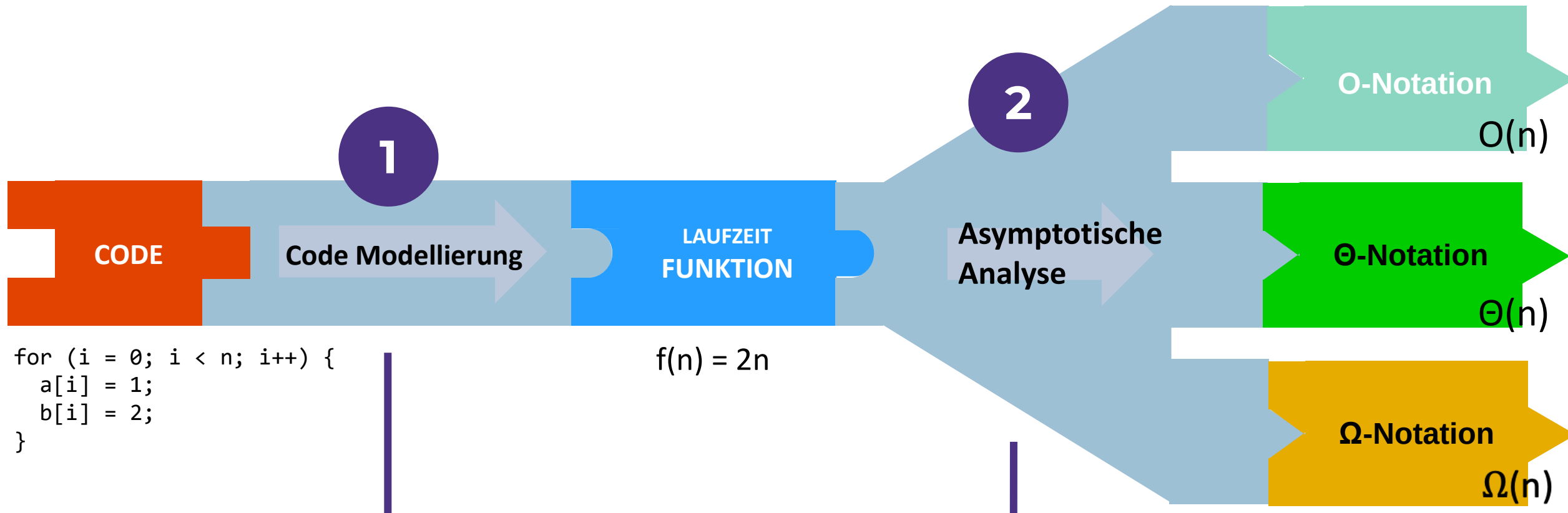
Unser verbessertes Werkzeug: Asymptotische Analyse



Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Übungsblatt 2 - Aufgabe 3, 7 und 10

Analyse von Algorithmen



Schauen wir uns dieses Werkzeug nun genauer an. Wie genau kommen wir zu dieser Funktion?

Wir haben gerade dieses Werkzeug fertiggestellt, um eine Funktion anhand einiger nützlicher Schranken zu charakterisieren!

Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

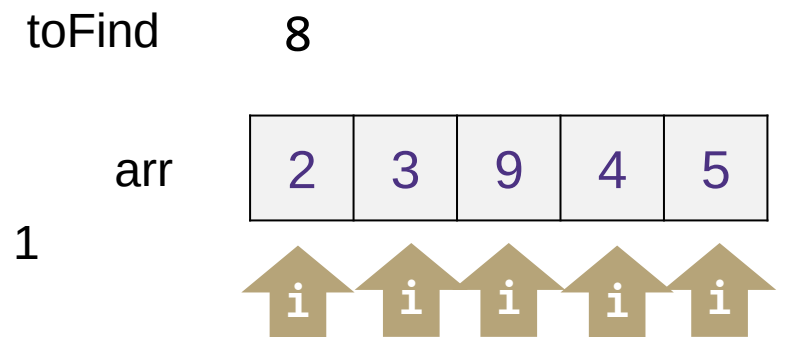
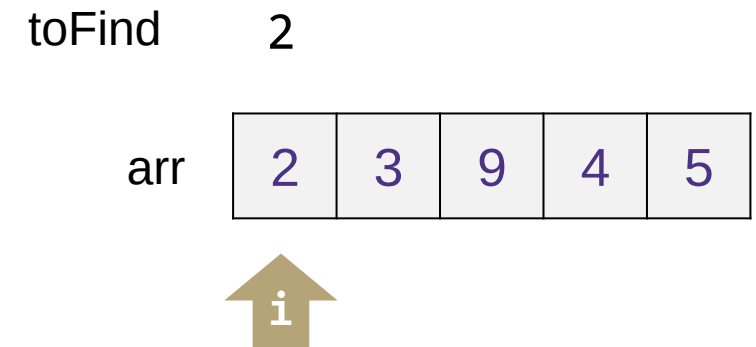
- Abstrakte Datentypen
 - Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
 - Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
 - Array
 - Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
 - O-Notation
- Laufzeitanalyse von Algorithmen durch Asymptotische Notation
 - O-Notation, Ω -Notation, Θ -Notation
 - **Fallanalyse: Worst Case, Best Case Analyse**

Fallanalyse für Lineare Suche

```
int linearSearch(int[] arr, int toFind) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == toFind)
            return i;
    }
    return -1;
}
```

Die Anzahl an Operationen hängt nicht nur von n (Größe des Arrays) ab. Es gibt ein paar Fälle, die es zu unterscheiden gilt:

- Falls $\text{toFind} == \text{arr}[0]$, benötigen wir nur eine Iteration: $f(n) = 4$
- Falls toFind nicht in arr ist, benötigen wir n Iterationen: $f(n) = 3n + 1$
- Viele weitere Fälle dazwischen

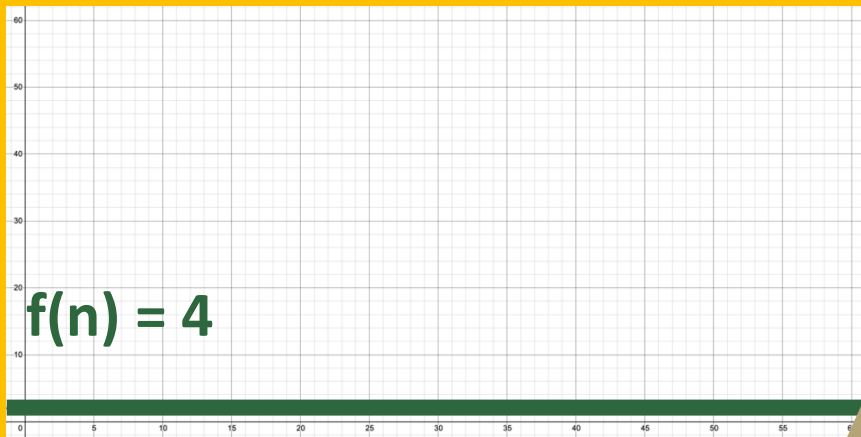


Best Case

On Lucky Earth

toFind 2

arr



Asymptotische Analyse:

$O(1)$

$\Theta(1)$

$\Omega(1)$

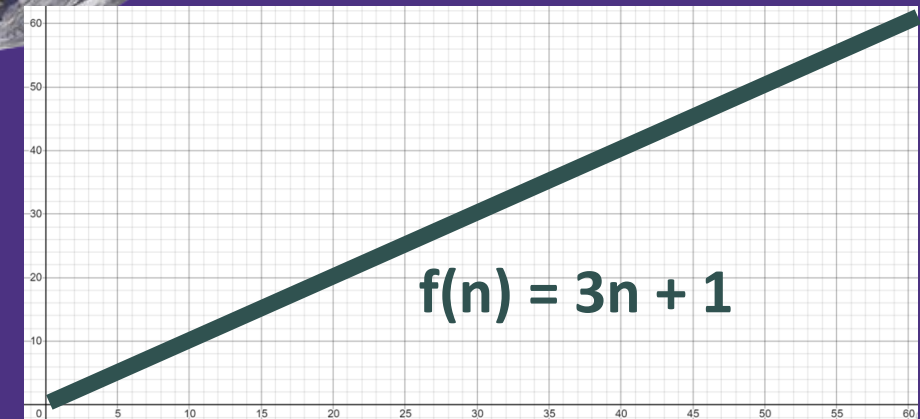


Worst Case

On Unlucky Earth (where it's 2020 every year)

toFind 8

arr



Asymptotische Analyse:

$O(n)$

$\Theta(n)$

$\Omega(n)$



Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2022

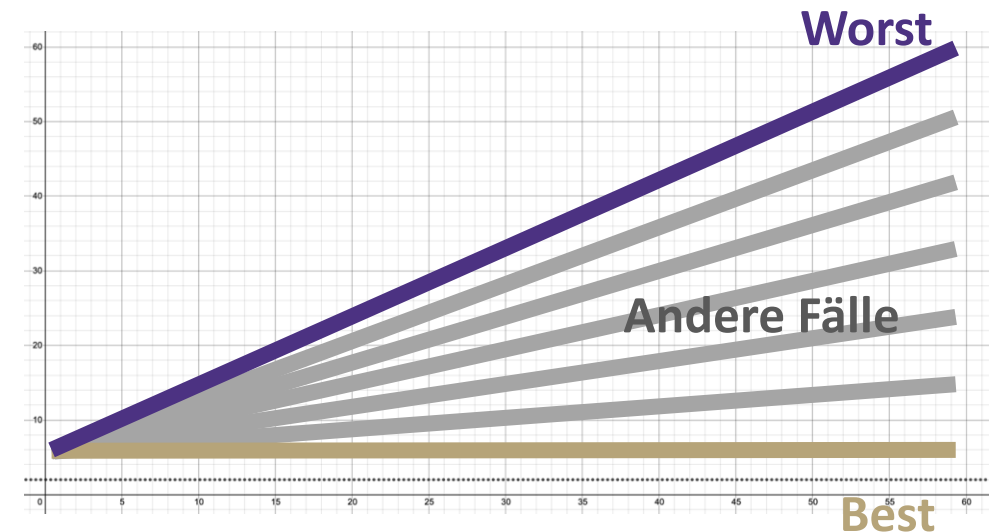
Fallanalyse – Welchen Mehrwert bietet diese?

- Ihr Code könnte in manchen Situationen sehr schnell sein und in anderen Situationen sehr langsam
- Wenn Sie dem User der Methode sagen können, in welchen Situationen die Methode schnell/langsam ist, ist das für den User eine hilfreiche Information
- Beispiel:
 - User muss sich zwischen Ihrer Implementierung einer Methode und einer anderen Implementierung entscheiden
 - User weiß, dass in der Anwendung in der sein Programm genutzt wird, fast immer Situation A auftritt und fast nie Situation B
 - Durch Ihre Fallanalyse können Sie dem User mitteilen, dass in Situation A Ihre Implementierung konstante Laufzeit hat und in Situation B quadratische Laufzeit
 - Hinweis: Ohne die Fallanalyse wäre die Laufzeit Ihrer Methode immer in $O(n^2)$ und in $\Omega(1)/\Omega(n^2)$
 - Der User kann Ihre Implementierung mit gutem Gewissen nutzen

Fallanalyse

Fall: eine Beschreibung der Eingabeparameter/Zustände eines Algorithmus, die spezifisch genug ist, um eine Laufzeitfunktion zu erstellen, deren einziger Parameter die Eingabegröße n ist

- Die Fallanalyse ist unser Werkzeug, um über **alle anderen Variationen als n** zu argumentieren!
- Tritt während der Code Modellierung und nicht während der Asymptotischen Analyse auf!
- (Best Case: schnellste/Worst Case: langsamste) Zeit, die unser Code bei einer Eingabegröße n benötigen könnte.
- Wichtig ist, dass jede Position von `toFind` in `arr` ein eigener Fall sein kann!
- Für dieses einfache Beispiel ist das egal (sie haben alle die obere Schranke $O(n)$)



Vorsicht! Häufige Missverständnisse

Der Best/Worst Case basiert auf allen Variationen außer dem Wert von n

Falsch - basierend auf bestimmten Werten von n

- „Der Best Case ist, wenn $n=1$, der Worst Case, wenn $n=\text{unendlich}$ “
- „Der Best Case ist, wenn der front Pointer Null ist“
- „Der Best Case ist, wenn overallRoot Null ist“ (overallRoot ist Wurzelknoten eines Baumes)
- „Der Best Case ist, wenn n eine gerade Zahl ist“

Richtig - basierend auf dem Zustand der Datenstruktur, unabhängig von n

- „Der Best Case ist, wenn der gesuchte Wert vorne steht, der Worst Case, wenn er nicht in der Liste ist“
- „Der Best Case ist, wenn Werte in der Liste sortiert sind, der Worst Case, wenn Liste unsortiert ist“
- „Der Best Case ist, wenn der binäre Suchbaum perfekt ausbalanciert ist, der Worst Case, wenn er aus einer einzigen Linie von Knoten besteht“

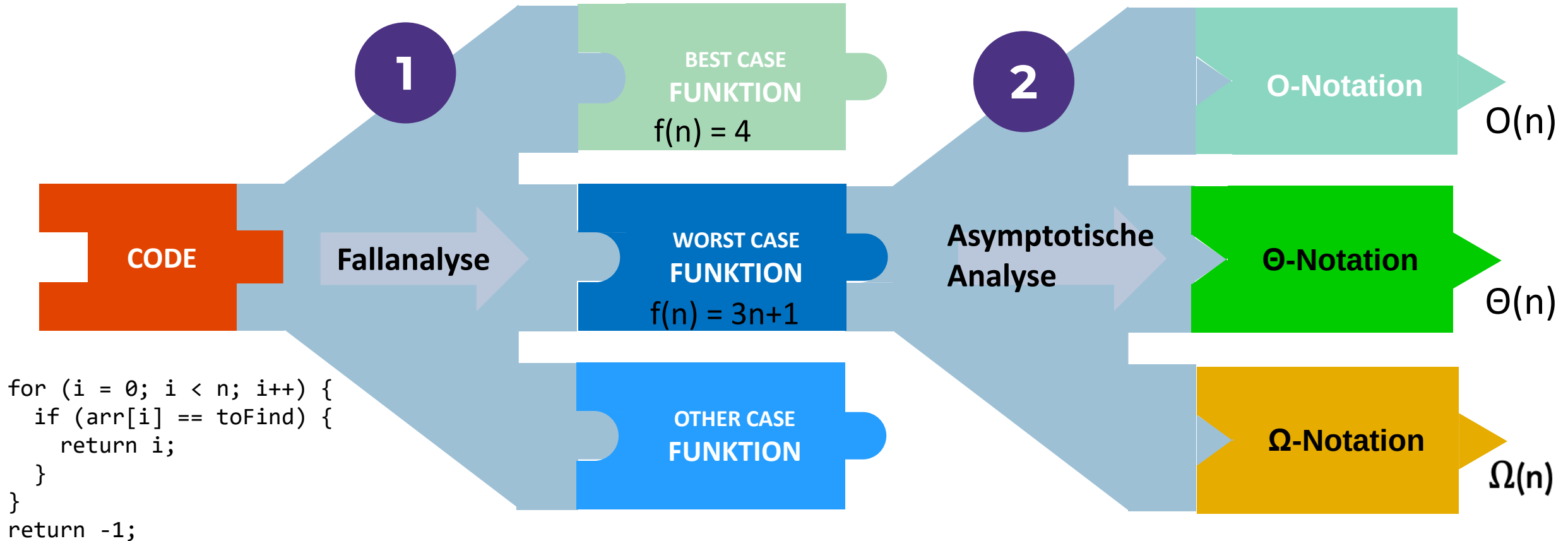
Wie Sie eine Fallanalyse durchführen

1. Sehen Sie sich den Code an und verstehen Sie, wie sich die Situation je nach Eingabe ändern kann.
 - Wie können Schleifen vorzeitig beendet werden?
 - Kann man vorzeitig zurückkehren (return) (die Methode verlassen)?
 - Sind einige if/else-Verzweigungen viel langsamer als andere?
2. Finden Sie heraus, welche Eingaben dazu führen können, dass die (besten/schlechtesten) Teile des Codes erreicht werden.
3. Laufzeitfunktion für diesen Fall bestimmen
4. Führen Sie nun die Asymptotische Analyse wie gewohnt durch!

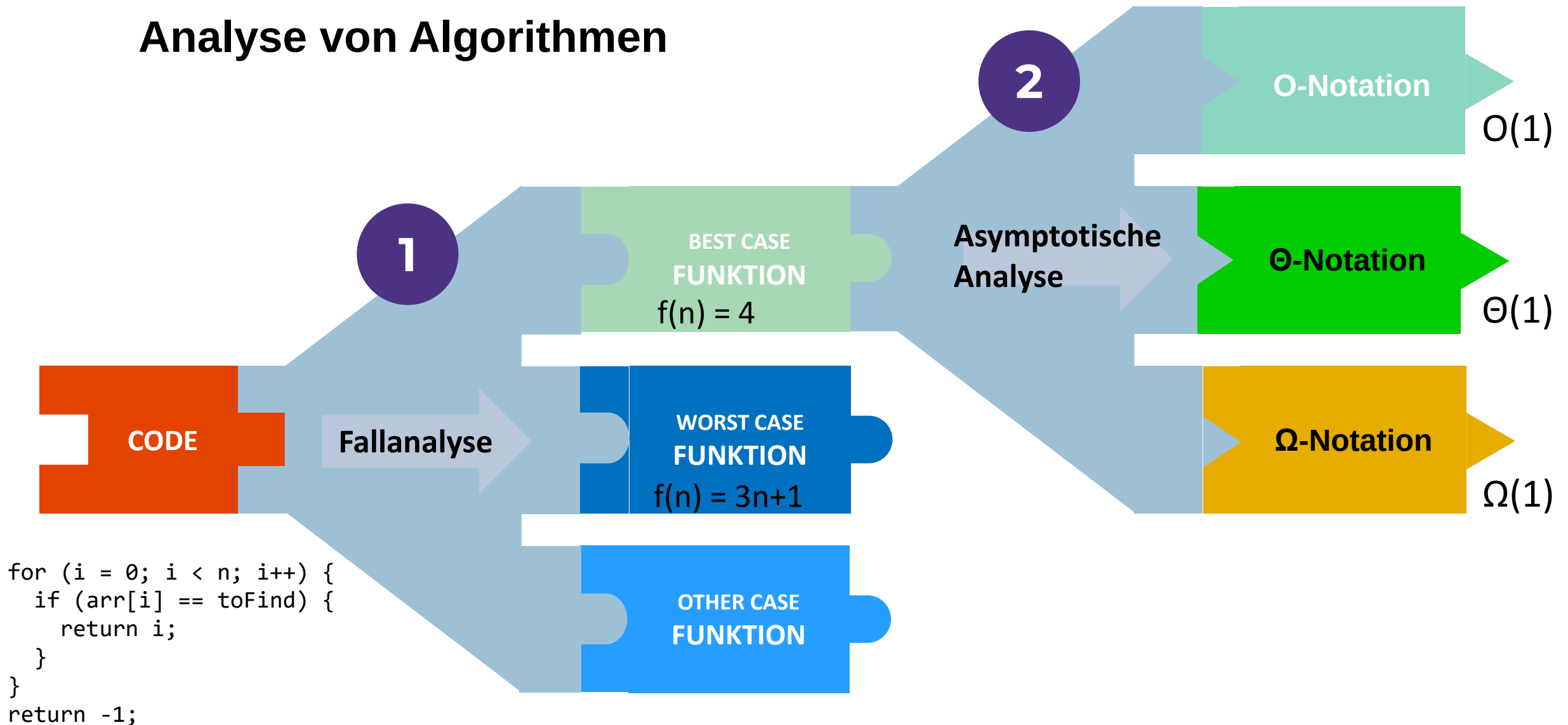
Fallanalyse: Andere Fälle

- Der „Angenommen X tritt nicht ein“ Fall (amortisiert):
 - Eine häufige Annahme ist, dass Größe von Array nicht verändert werden muss
- Durchschnittliche Fall:
 - Annahme, dass die Eingangsdaten zufällig verteilt sind
 - Sie müssen angeben, welche Eingaben möglich sind und wie wahrscheinlich sie sind
 - $f(n)$ ist nun die durchschnittliche Anzahl von Operationen bei einer zufälligen Eingabe der Größe n
- „In der Praxis“ Fall:
 - Dies ist kein richtiger Begriff
 - Machen Sie einige vernünftige Annahmen darüber, welche Daten/Situationen in der Praxis wahrscheinlich zu erwarten sind
 - In dieser Vorlesung teilen wir Ihnen die Annahmen mit
 - Führen Sie dann die Asymptotische Analyse unter diesen Annahmen durch
- Alle diese Fallanalysen können mit den Notationen O , Ω und Θ kombiniert werden!

Analyse von Algorithmen



Analyse von Algorithmen



```
for (i = 0; i < n; i++) {
    if (arr[i] == toFind) {
        return i;
    }
}
return -1;
```

Fallanalyse

Fall	O-Notation	Ω -Notation	Θ -Notation
Worst Case	Bei der schlechtesten Eingabe, wenn n größer wird, benötigt der Code höchstens so viel Zeit	Bei der schlechtesten Eingabe, wenn n größer wird, benötigt der Code mindestens so viel Zeit	Bei der schlechtesten Eingabe, wenn n größer wird, benötigt der Code genau so viel Zeit (bis auf Konstanten)
Best Case	Bei der besten Eingabe, wenn n größer wird, benötigt der Code höchstens so viel Zeit	Bei der besten Eingabe, wenn n größer wird, benötigt der Code mindestens so viel Zeit	Bei der besten Eingabe, wenn n größer wird, benötigt der Code genau so viel Zeit (bis auf Konstanten)

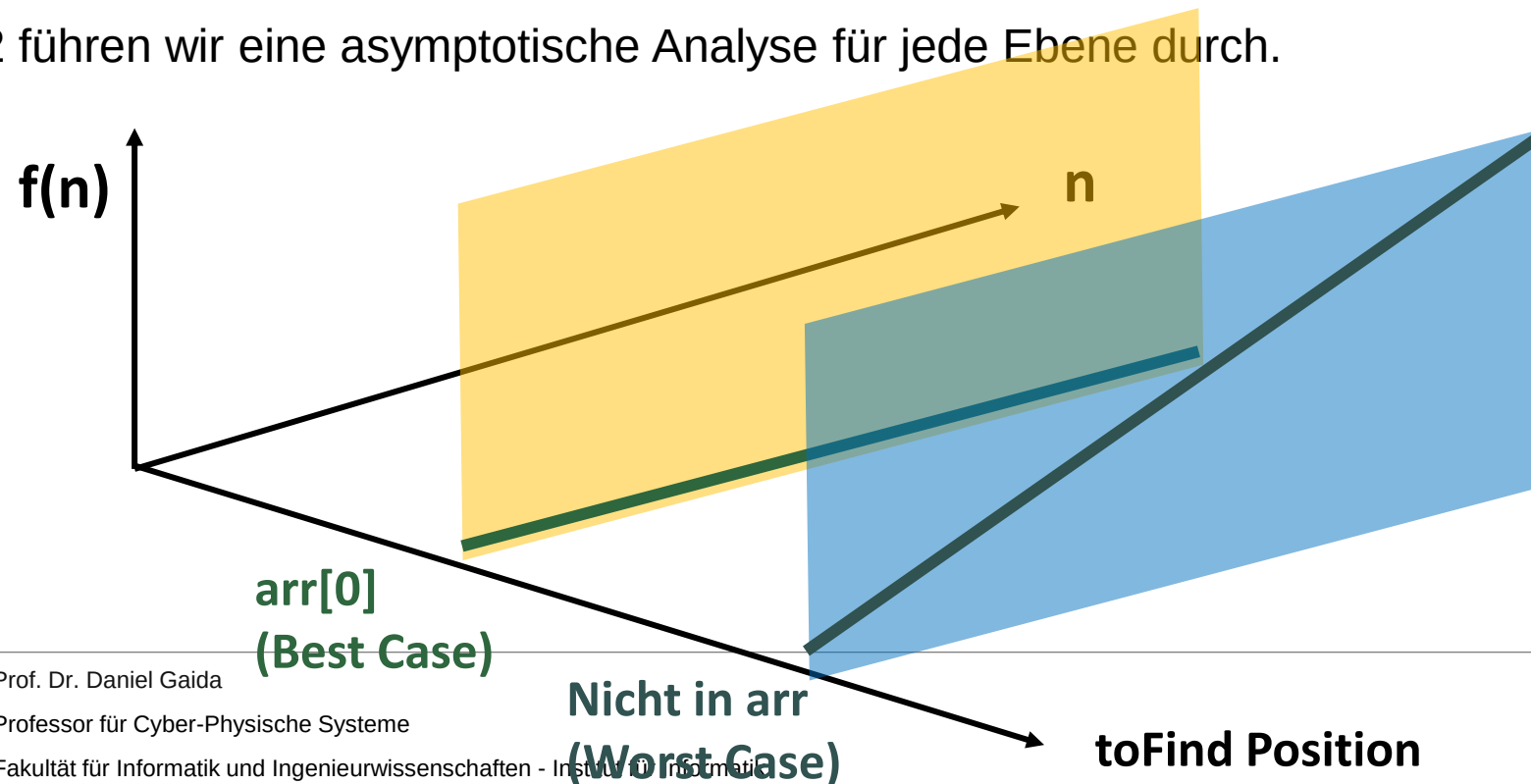
„schlechteste Eingabe“: Eingabe, die den Code am langsamsten ausführen lässt.

Fallanalyse als 3-dimensionales Diagramm

Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2022

Stellen Sie sich ein 3-dimensionales Diagramm vor

- Der Fall, den wir betrachten, ist eine Dimension.
- Durch die Wahl eines Falles können wir einen „Ausschnitt“ der anderen Dimensionen nehmen: n und $f(n)$
- In Schritt 2 führen wir eine asymptotische Analyse für jede Ebene durch.



Fallanalyse vs. $O/\Omega/\Theta$ -Notation

Hängt Laufzeit eines Algorithmus von einer Eigenschaft von n ab?

- n (un-)gerade, Primzahl,
- $\rightarrow O/\Omega/\Theta$ -Notation
- Bsp.: Algorithmus `isPrime()`

Hängt Laufzeit eines Algorithmus von irgendetwas anderem als n ab?

- Argumente des Algorithmus abgesehen von n ,
- Werte in einer Datenstruktur,
- die dazu führen, dass unterschiedliche Teile des Algorithmus ausgeführt werden
 - if/else, break, return, ...
- $\rightarrow$ Fallanalyse

Fallanalyse vs. $O/\Omega/\Theta$ -Notation

```
def f(n):  
    if n < 5:  
        fib(n)  
    else:  
        print(„Hello world!“)
```

```
def f(n):  
    if n % 2 == 0:  
        fib(n)  
    else:  
        print(„Hello world!“)
```

fib(n) berechnet die nte Fibonacci Zahl und
Laufzeitfunktion von fib(n) ist in $O(2^n)$

$f(n) \in O(1)$

$f(n) \in \Omega(1)$

$f(n) \in \Theta(1)$

$f(n) \in O(2^n)$

$f(n) \in \Omega(1)$

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können das asymptotische Laufzeitverhalten eines Algorithmus angeben, indem sie aus dem Code eine Best Case und eine Worst Case Laufzeitfunktion bestimmen und die O -, Ω - und Θ -Notation auf die Laufzeitfunktionen anwenden, um später die Laufzeit komplexer Algorithmen bestimmen zu können und dadurch bei Vorlage mehrerer Algorithmen den effizientesten Algorithmus auswählen zu können.

s. Übungsblatt 2

Übung – Welche der folgenden Aussagen sind korrekt?

- Für jede Laufzeitfunktion existiert eine Θ -Notation.
- Ein möglicher Best Case für das Hinzufügen zu ArrayQueue ist, wenn sie leer ist.
- Ein möglicher Best Case für das Hinzufügen zu ArrayQueue ist, wenn in dem Array Platz ist.
- Wir nutzen die Ω -Notation nur für eine Worst Case Analyse.
- Wenn eine Laufzeitfunktion in $O(n^2)$ ist, kann sie nicht auch in $\Omega(n^2)$ sein.

O-, Ω - und Θ -Notation

Übung

```
public void printSmilies(ArrayList<Integer> input, int target) {
    LinkedList<Integer> products = new LinkedList<>();
    for (int i = 0; i < input.size(); i++) {
        for (int j = 0; j < input.size(); j++) {
            products.add(input.get(i) * input.get(j));
        }
    }
    LinkedList<Integer> occurrencesOfToFind = new LinkedList<>();

    while (products.size() != 0) {
        int next = products.remove(0);
        if (next == target) {
            occurrencesOfToFind.add(next);
        }
    }

    for (int i = 0; i < occurrencesOfToFind.size(); i++) {
        for (int j = 0; j < occurrencesOfToFind.size(); j++) {
            System.out.println(":D");
        }
    }
}
```

Annahme, dass
add() am Anfang
der Liste einfügt



Nächste Vorlesung und Fragen

- Laufzeitanalyse rekursiver Algorithmen
- Fragen?